

Chapter 2

THE BUFFER CACHE

TOPICS

- Buffer Header
- Structure of the Buffer Pool
- Scenarios for Retrieval of Buffer
- Reading and Writing Disk Blocks
- Advantages and disadvantages of the buffer cache

Introduction: Buffer Cache

- The kernel attempts to minimize the frequency of disk access
- Buffer cache == pool of buffers
- Buffer cache contains the data in recently used disk blocks
- Higher-level kernel algorithms instruct the buffer cache modules to pre-cache data or to delay-write data to maximize caching effect

2.1 Buffer Header

- Buffer consists of two parts:
 - Buffer header: identifies the buffer
 - Memory array: contains disk block data
- A disk block can never map into more than one buffer at a time
- Buffer status: combination of following
 - Locked(or busy) $\Leftrightarrow$ Unlocked(or free)
 - Buffer is valid
 - Write buffer contents before reassigning buffer
 - Currently reading or Writing the contents to disk
 - Currently waiting for the buffer to become free

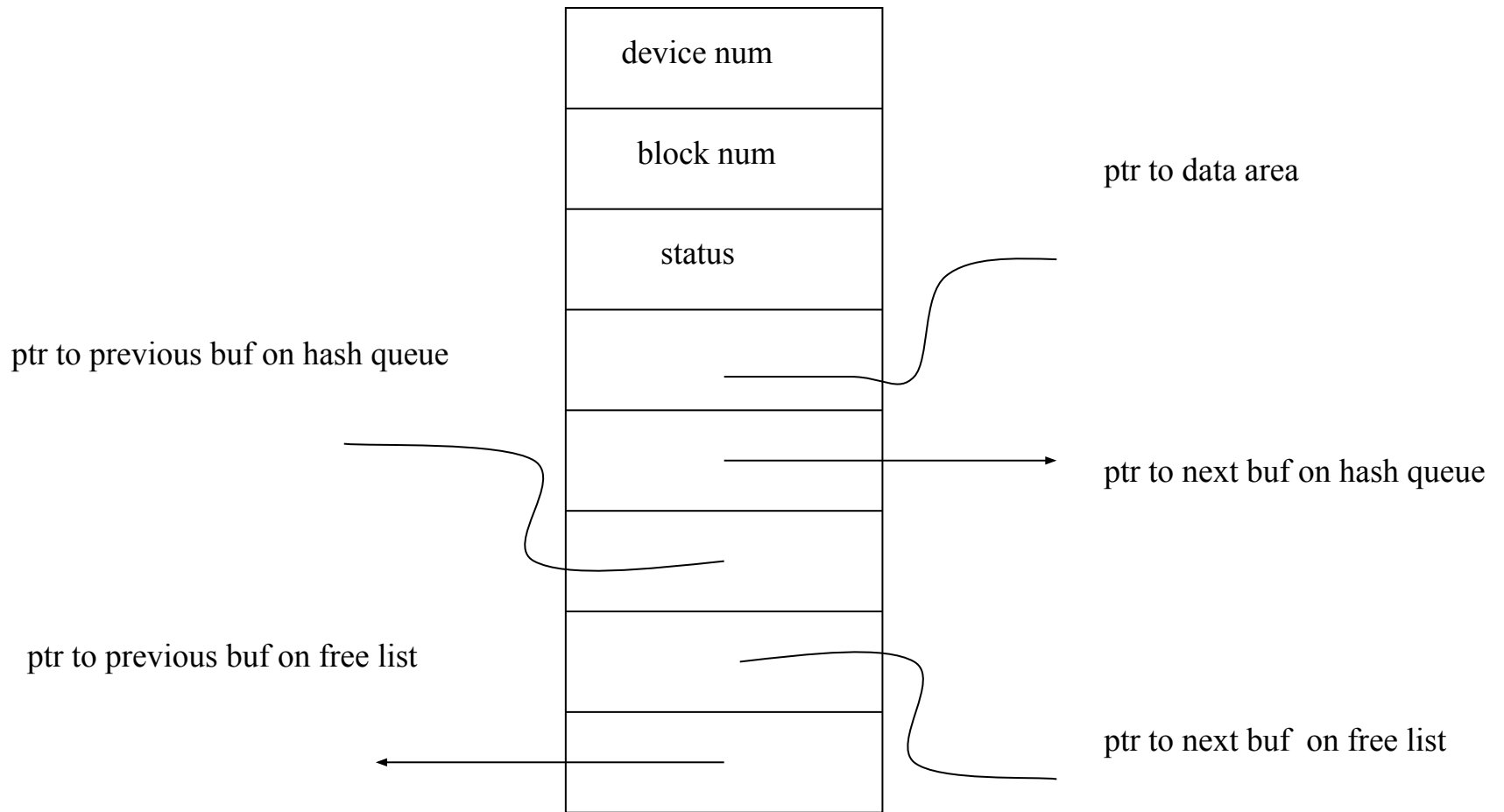


Figure 2.1 Buffer Header

2.2 Structures of the buffer pool

- Buffer pool according to LRU
- The kernel maintains a free list of buffer
 - doubly linked list
 - take a buffer from the head of the free list.
 - When returning a buffer, attaches the buffer to the tail.

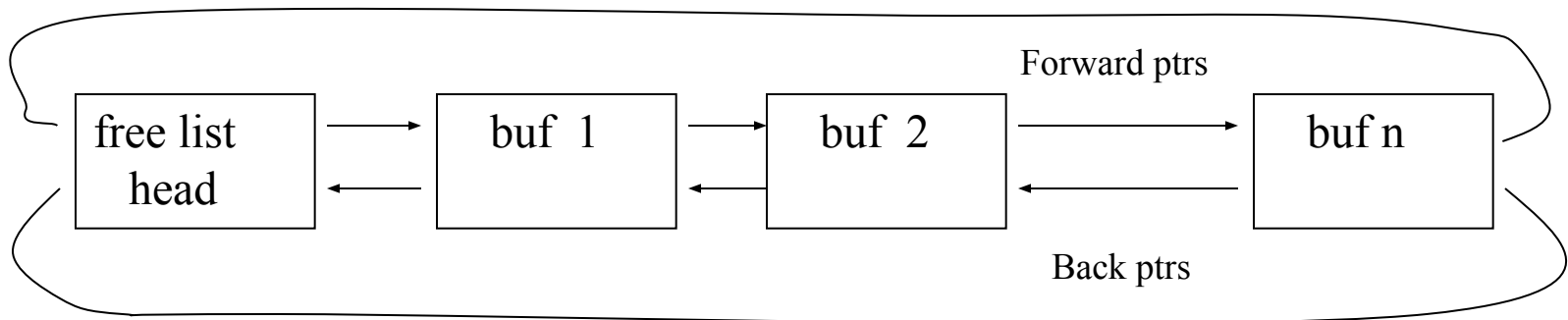


Figure 2.2 Free list of Buffers

Cont....

- When the kernel accesses a disk block
 - separate queue (doubly linked circular list)
 - hashed as a function of the device and block num
 - Every disk block exists on one and only on hash queue and only once on the queue

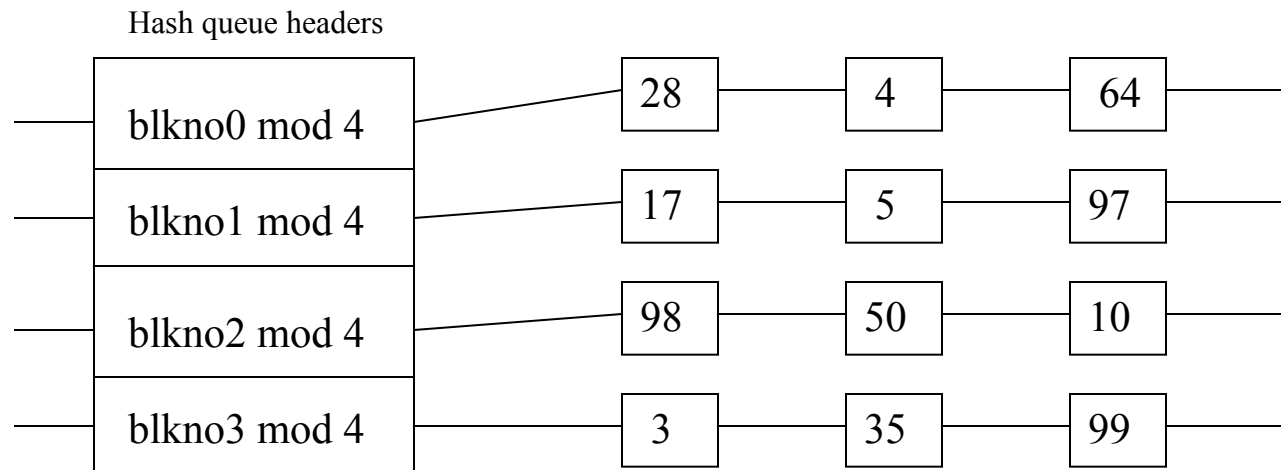


Figure 2.3 Buffers on the Hash Queues

2.3 Scenarios for retrieval of a buffer

- Five typical scenarios in `getblk()`
 1. Block in the hash queue, and its buffer is free
 2. Cannot find block on the hash queue =>
allocate a buffer from free list
 3. Scenario 2, but buufer on the free list marked “delayed write” => flush “delayed write” buffer and allocate another buffer
 4. Cannot find block on the hash queue and free list of buffer also empty
 5. Block in the hash queue, but buffer is busy

Algorithm for buffer allocation

Algorithm getblk

Input: File system number

block number

Output: locked buffer that can now be used for block

while(buffer not found)

{

if(block in hash queue)

{

if(buffer busy) /*scenario 5*/

{

sleep(event buffer becomes free);

continues; /*back to while*/

}

mark buffer busy; /*scenario 1*/

remove buffer from free list;

return buffer;

}

/* Next Page... */

Algorithm for buffer allocation

```
else
{
    if(there are no buffer on free list)      /*scenario 4 */
    {
        sleep(event any buffer becomes free);
        continue;      /*back to while loop*/
    }
    remove buffer from free list;
    if(buffer marked for delayed write) {    /*scenario 3 */
        asynchronous write buffer to disk;
        continue;      /*back to while loop*/
    }
    /* scenario 2 – found a free buffer */
    remove buffer from old hash queue;
    put buffer onto new hash queue;
    return buffer;
}
}
```

Scenarios for retrieval of a buffer

- The first scenario-1

Hash queue headers

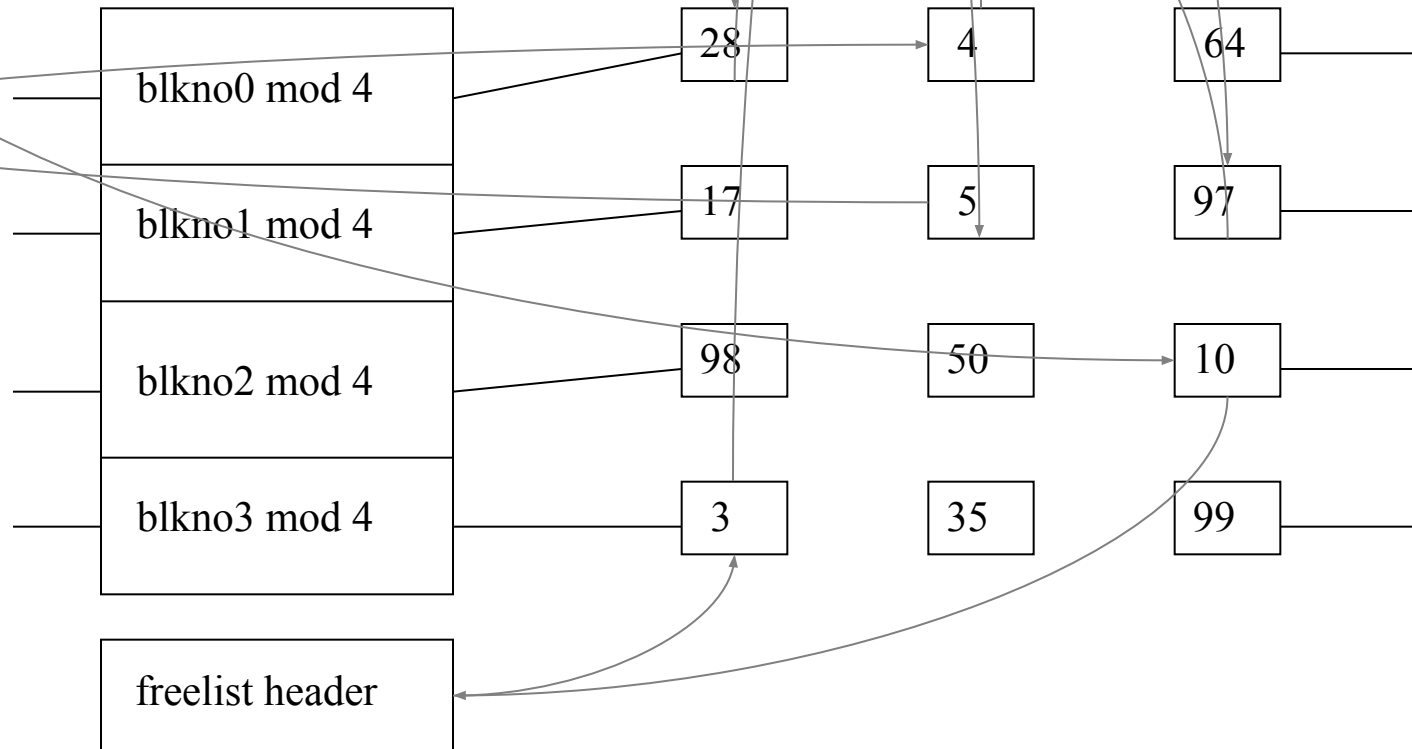
Diagram illustrating the search for Block 4 on the First Hash Queue. The diagram shows a hash table with four slots: blkno0 mod 4, blkno1 mod 4, blkno2 mod 4, and blkno3 mod 4. Each slot points to a linked list of blocks. The search path for Block 4 is highlighted: blkno0 mod 4 points to 28, which points to 4. The freelist header also points to 4. The diagram is labeled (a) Search for Block 4 on First Hash Queue.

(a) Search for Block 4 on First Hash Queue

Figure 2.5

- The first scenario-1

Hash queue headers

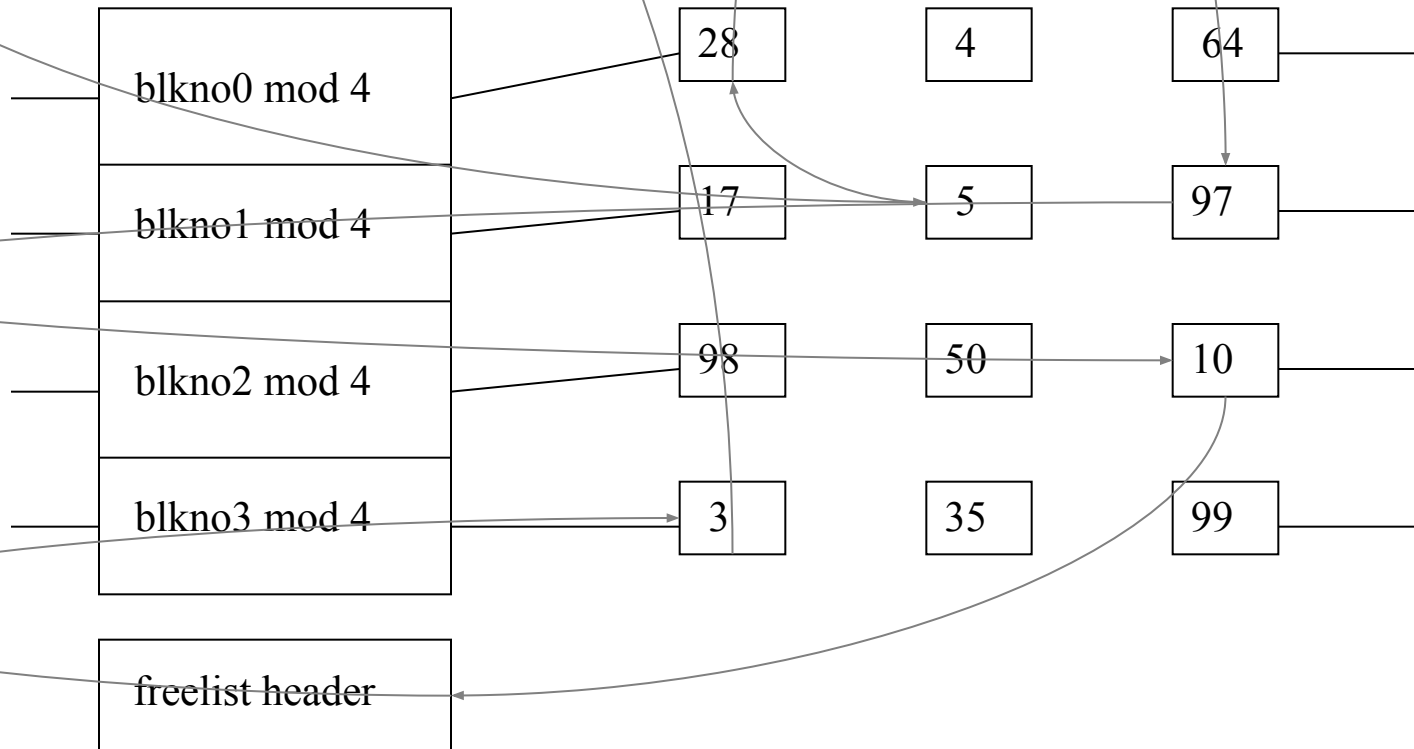


(a) Search for Block 4 on First Hash Queue
Figure 2.5

Cont....

- The first scenario-2

Hash queue headers



(b) Remove Block 4 from Free List
Figure 2.5

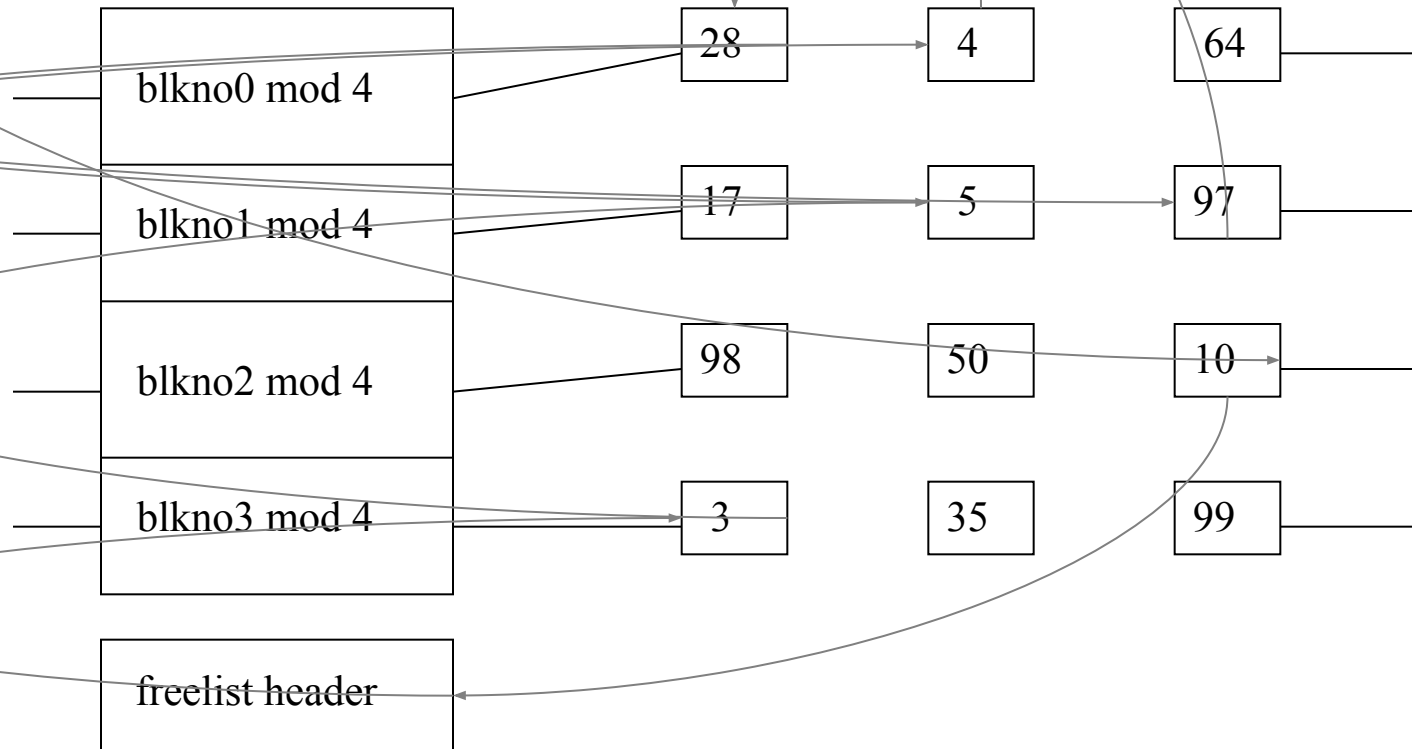
Algorithm for Releasing a Buffer

```
algorithm brelse
input:    locked buffer
output:   none
{
    wakeup all procs: event, waiting for any buffer to become free;
    wakeup all procs: event, waiting for this buffer to become free;
    raise processor execution level to block interrupts;
    if(buffer contents valid and buffer not old)
        enqueue buffer at end of free list
    else
        enqueue buffer at beginning of free list
    lower processor execution level to allow interrupts;
    unlock(buffer);
}
```

Scenarios for retrieval of a buffer

- The second scenario-1

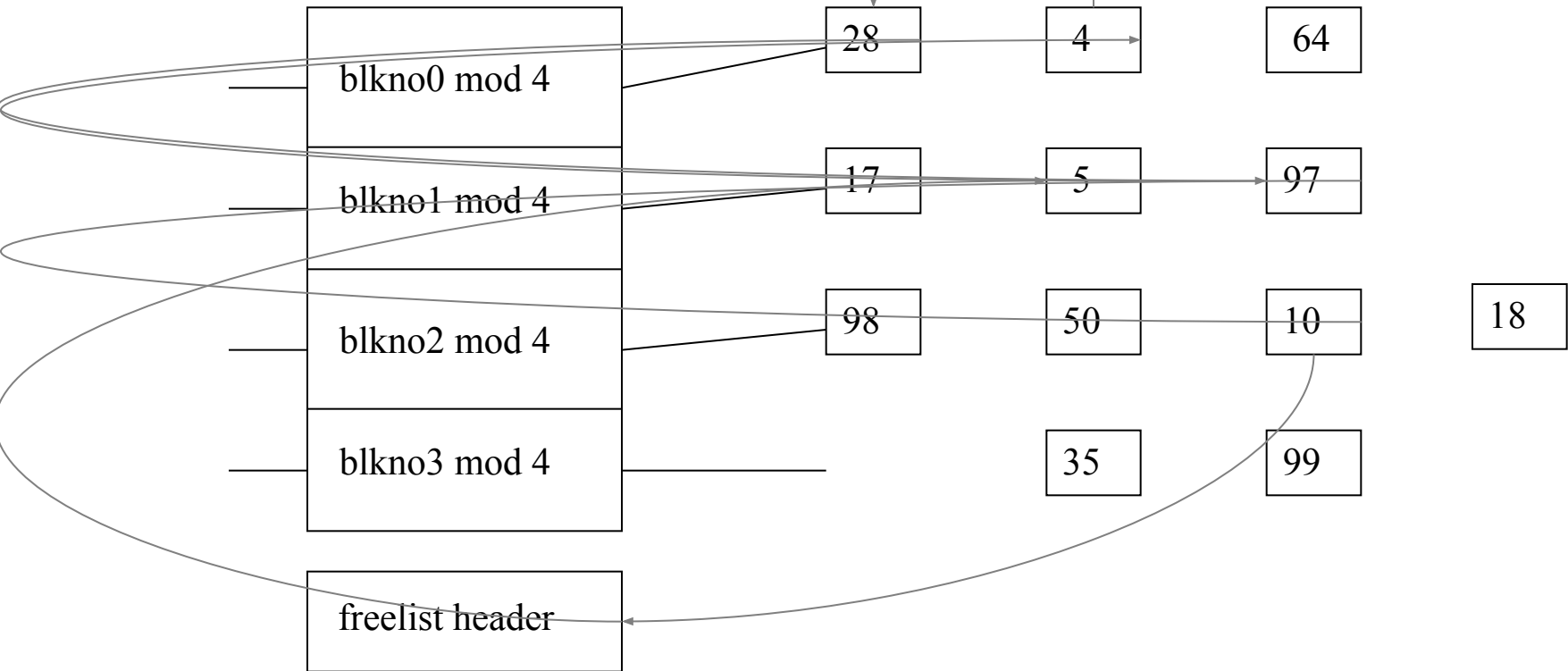
Hash queue headers



(a) Search for Block 18-Not in Cache
Figure 2.7

- The second scenario-2

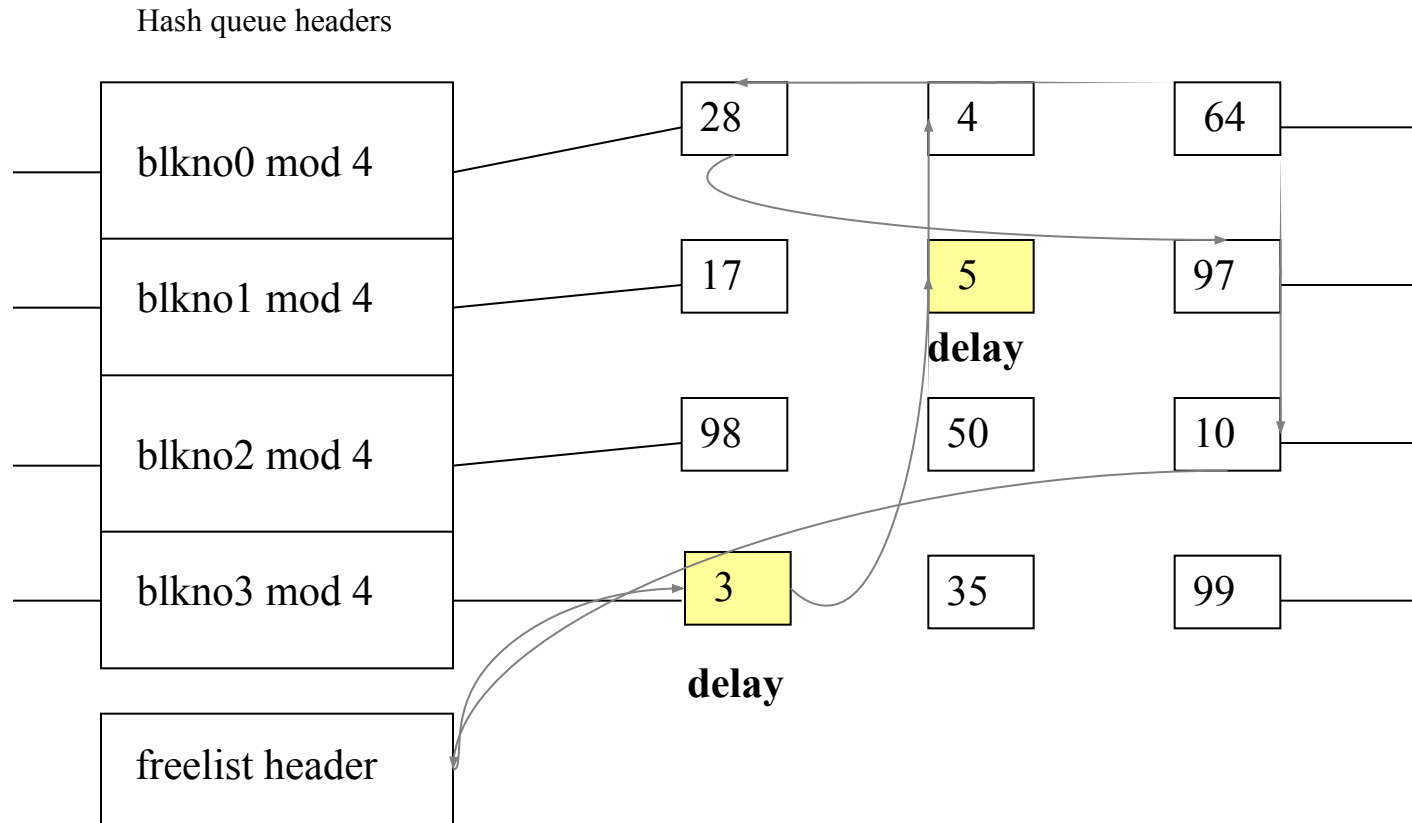
Hash queue headers



(b) Remove First Block from Free List, Assign to 18

Scenarios for retrieval of a buffer

- The third scenario-1

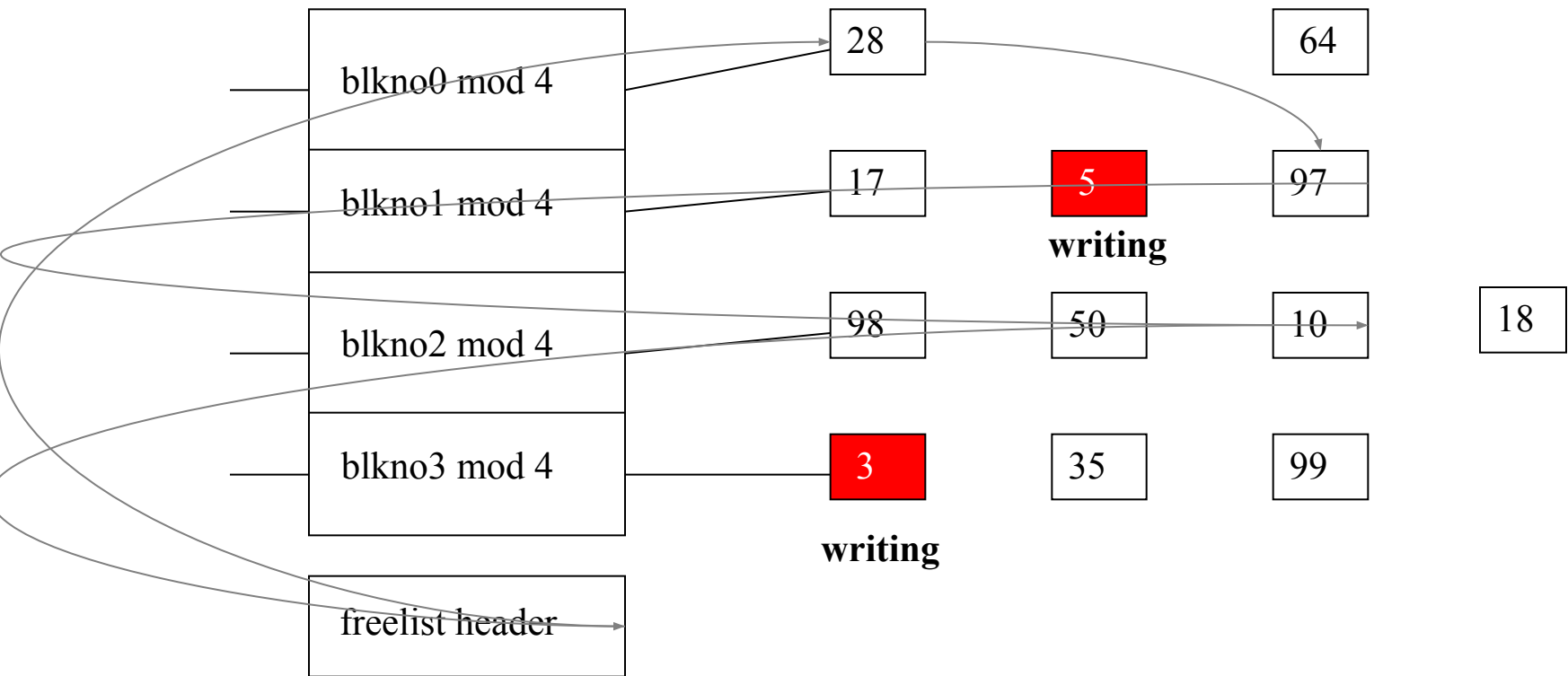


(a) Search for Block 18- Delayed Write Blocks on Free List
Figure 2.8

Scenarios for retrieval of a buffer

- The third scenario-2

Hash queue headers



(b) Writing Blocks 3, 5, Reassign 4 to 18
Figure 2.8

Scenarios for retrieval of a buffer

- The fourth scenario

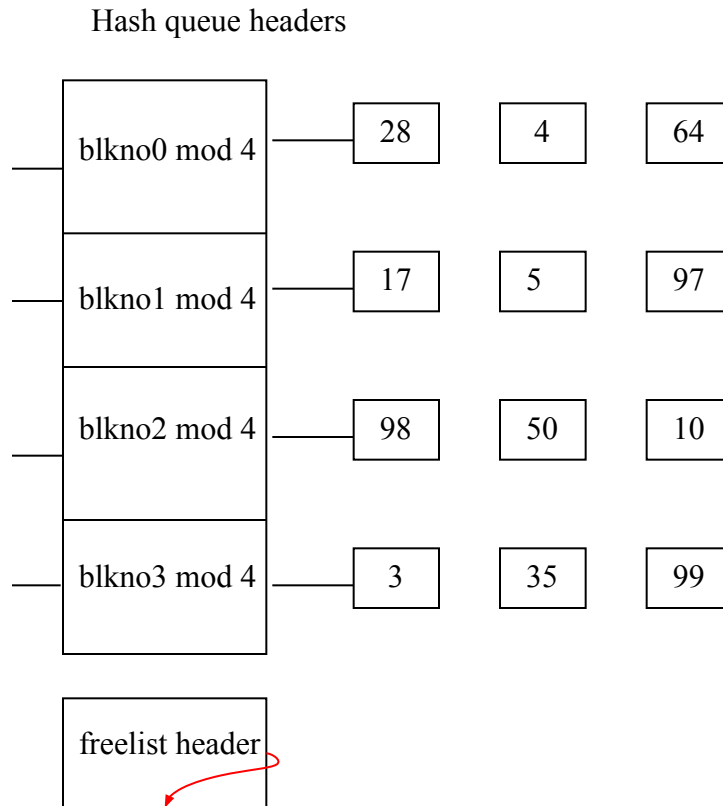


Figure 2.9 Fourth Scenario for Allocating Buffer

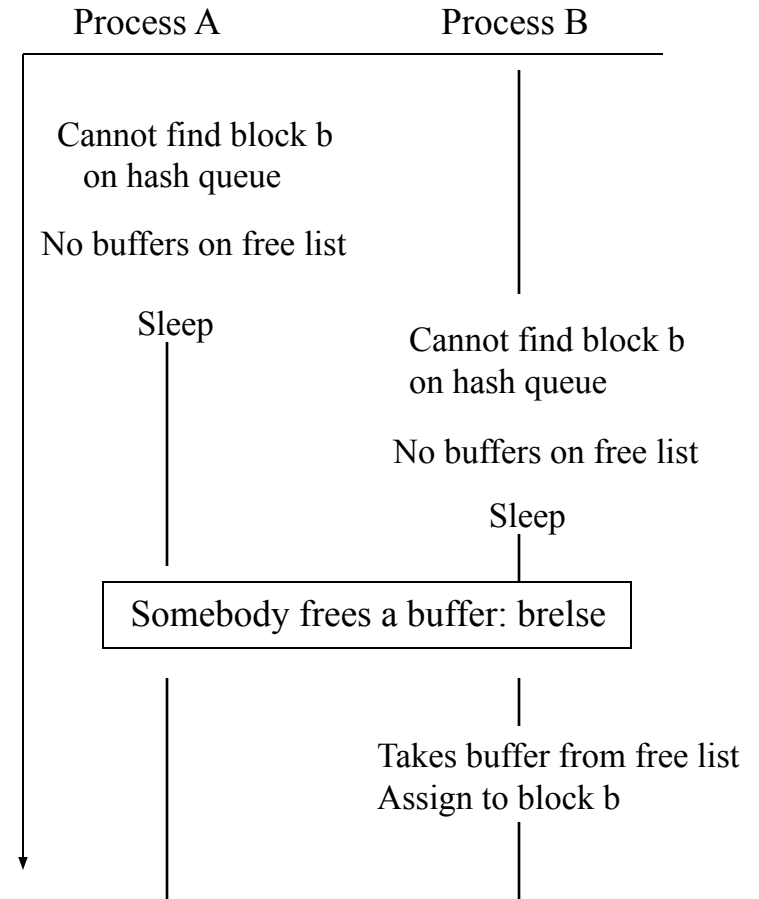


Figure 2.10 Race for Free Buffer

Scenarios for retrieval of a buffer

- The fifth scenario

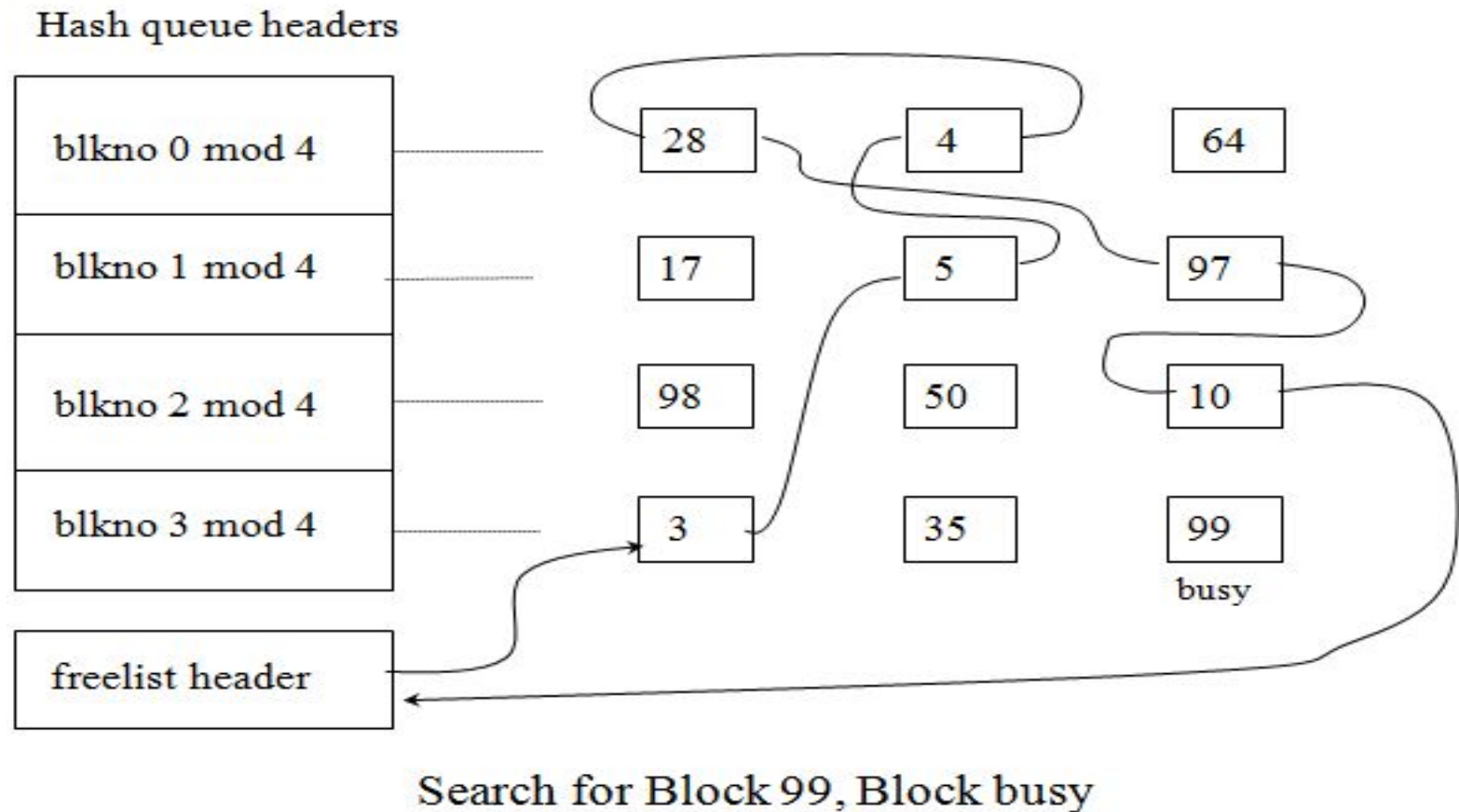


Figure 2.11 fifth Scenario for Allocating Buffer

Scenarios for retrieval of a buffer

- The fifth scenario

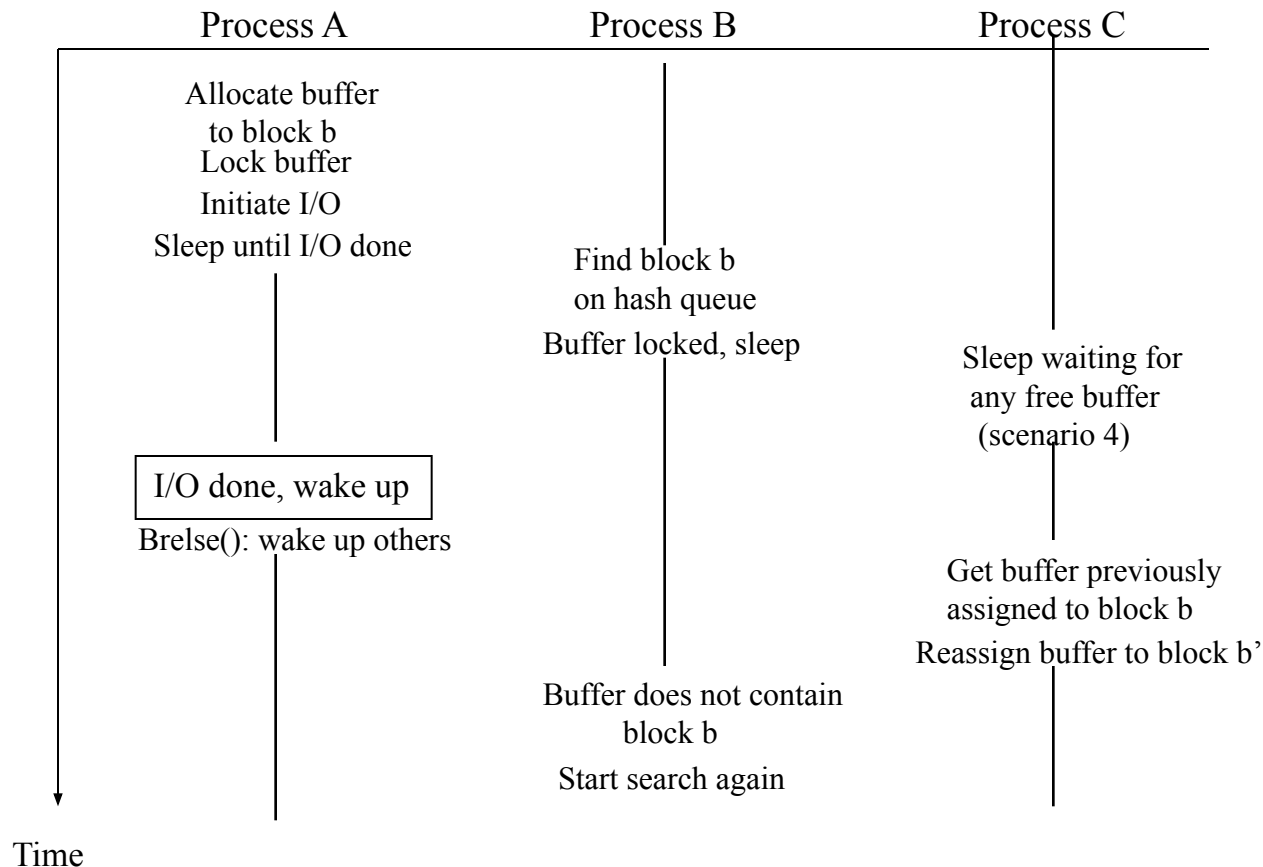


Figure 2.12 Race for a Locked Buffer

2.4 Reading a disk block

```
algorithm bread /*block bread*/  
input: file system block number  
output: buffer containing data  
{  
    get buffer for block(algorithm getblk);  
    if(buffer data valid)  
        return buffer;  
    initiate disk read;  
    sleep(event disk read complete);  
    return(buffer);  
}
```

Read Ahead

Algorithm breada

```
{
  if(first block not in cache)
  {
    get buffer for first block(algorithm getblk);
    if(buffer data not valid)
      initiate disk read;
  }
  if(second block not in cache)
  {
    get buffer for second block(algorithm getblk);
    if(buffer data valid)
      release buffer(algorithm brelse);
    else
      initiate disk read;
  }
```

Read Ahead(...)

```
if(first block was originally in cache)
{
    read first block(algorithm bread);
    return buffer;
}
sleep(event first buffer contains valid data);
return buffer;
}
```

Writing a disk block

```
algorithm bwrite /*block write*/
input:      buffer
output: none
{
    initiate disk write;
    if(I/O synchronous)
    {
        sleep(event I/O complete);
        release buffer(algorithm brelse);
    }
    else if(buffer marked for delayed write)
        mark buffer to put at head of free list;
}
```

2.5 Advantages and Disadvantages of the buffer cache

Advantages of the buffer cache

- Uniform disk access => system design simpler
- Copying data from user buffers to system buffers => eliminates the need for special alignment of user buffers.
- Use of the buffer cache can reduce the amount of disk traffic.
- Single image of disk blocks contained in the cache => helps insure file system integrity

Disadvantages of the buffer cache

- Delayed write => vulnerable to crashes that leave disk data in incorrect state
- An extra data copy when reading and writing to and from user processes => slow down when transmitting large data